

4주차 머신러닝 & 딥러닝 모의면접 핵심 정리 (구두 답변용)

이 문서는 구두 면접을 위한 지식 정리입니다. 수식 없이, 입으로 말할 수 있는 문장으로만 구성했습니다. 모범답안이 아니라 답을 내 언어로 조립하기 위한 재료입니다.

각 항목은 다음 순서로 구성됩니다.

1. 🎯 반드시 입에 올려야 할 키워드
2. 💡 핵심 원리 (비유·숫자 예시)
3. 🧠 심화 지식 (답변 재료) — 원리의 "왜", 반론, 실무 함정
4. 🔍 꼬리 질문과 대응 포인트
5. 🗨️ 용어를 말로 풀면

⚠️ 2026-09-21 검토 반영 (원본 문서 오류 정정): §2 Z-score의 한계, §7 "증강 없음"의 정확한 의미, §9 편향 보정이 없을 때의 실제 증상, §10 "100% AdamW" 과장 — 4곳을 고쳤습니다. ✅ 정정 표시로 남겨 두었습니다.

1. SVM과 Random Forest의 스케일링 영향도 차이

질문: SVM과 Random Forest에 동일한 데이터를 넣을 때, 스케일링(정규화/표준화) 여부가 두 모델의 성능에 미치는 영향이 왜 다른지 설명해 주세요.

🎯 반드시 입에 올려야 할 키워드:

- ☐ 거리 기반 모델 (SVM): 유클리드 거리와 마진 계산에 의존
- ☐ 스케일 지배 현상: 단위가 큰 피처가 결정 경계를 압도적으로 지배
- ☐ 트리 기반 모델 (Random Forest): 피처 하나씩, 기준값 하나로 자르는 분할
- ☐ 스케일 불변성: 크고 작음의 순서에만 의존하므로 단조 변환에 불변

💡 핵심 원리:

- **SVM (스케일링 안 하면 치명적):**
 - SVM은 데이터 사이의 기하학적 거리를 재서, 두 클래스를 가장 넓게 가르는 '마진'을 찾는다.
 - 숫자 예시 (집값 vs 방 개수): A집은 집값 10억에 방 3개, B집은 집값 9억에 방 2개. 두 집의 거리를 재면 집값 차이 1억이 거리 전체를 차지하고, 방 개수 차이 1개는 그 앞에서 사실상 0이 된다. 결국 SVM은 방 개수를 아예 안 보고 집값만으로 분류한다.
- **Random Forest (스케일링 전혀 필요 없음):**
 - 트리는 거리를 재지 않는다. 피처 하나를 잡고 "이 값이 기준보다 큰가 작은가"라는 스무고개만 던진다.
 - 비유 (키 순서 줄세우기): 학생들을 키 순서로 세우고 "170cm 이상은 오른쪽"으로 나눈다. 단위를 cm에서 m로 바꾸든, 100을 곱하든, 로그를 씌우든 **줄 서 있는 순서는 절대 안 바뀐다**. 트리는 순서만 보므로 분할 결과가 100% 동일하다. 기준선 숫자만 같이 바뀔 뿐.

🧠 심화 지식 (답변 재료):

- **SVM이 스케일에 종속되는 지점은 두 곳이다:**
 1. 거리·커널: RBF 커널은 두 점 사이 거리로 유사도를 계산한다. 거리 안에서 단위 큰 피처가 지배한다.
 2. 규제항 자체: SVM은 "가중치가 큰 것을 벌주는" 원칙으로 마진을 넓힌다. 그런데 피처 단위가 바뀌면 같은 경계선을 그리기 위해 필요한 가중치 크기가 달라진다. 집값 단위를 '원'에서 '억 원'으로 바꾸면 집값 가중치가 1

억 배 커져야 같은 선이 나오는데, SVM은 큰 가중치를 피하려 하니 **다른 경계선을 고른다**. 즉 커널이 없는 선형 SVM도 스케일에 종속이다.

- → "속도 문제가 아니라 **답 자체가 달라지는 문제**"라고 구분해서 말하면 강하다.

- **트리가 불변인 이유를 정확히:**

- 트리의 분할 후보는 항상 "어떤 피처가 어떤 기준값보다 작거나 같은가"뿐이다. 이 질문이 데이터를 어떻게 두 덩어리로 나누는지는 **그 피처 값들의 순서**로만 결정된다.
- 순서를 보존하는 변환(단조 변환)을 하면 가능한 분할 목록이 그대로이고, 각 분할의 불순도 감소량도 그대로이므로 **완전히 같은 트리**가 나온다.
- 부트스트랩이나 피처 서브샘플링(RF의 장치)도 이 성질을 깨지 않는다. LightGBM의 히스토그램 비닝도 분위 수 기반이라 불변.
- 주의: 순서를 바꾸는 변환(음수가 섞인 제곱, 사인 함수 등)은 불변이 아니다.

- **모델별 스케일링 필요 여부 (말로 외우기):**

- **필요한 쪽** — "거리·분산·규제·경사하강" 네 단어:
 - 거리: KNN, K-Means, SVM
 - 분산: PCA (분산 큰 피처가 주성분을 독점)
 - 규제: Ridge, Lasso, 규제 있는 로지스틱 (벌금이 스케일에 의존)
 - 경사하강: 신경망 전반 (수렴 속도)
- **불필요한 쪽** — "순서만 보는 것": 결정트리, RF, XGBoost, LightGBM, CatBoost. 그리고 피처별 독립 확률을 쓰는 Naive Bayes.
- 예외: XGBoost의 선형 부스터는 필요. 피처를 선형결합해 대각선으로 자르는 Oblique tree는 필요. 스택킹 메타모델이 선형/NN이면 필요.
- **SVM에 표준화 vs 정규화?** 둘 다 가능하지만 보통 표준화. RBF 커널의 감마 값이 거리 크기에 따라 의미가 달라지기 때문에, 표준화해 두면 기본 감마가 잘 맞는다.

🔍 꼬리 질문과 대응 포인트:

- **Q1. "트리 기반 모델이라도 스케일링이 필요한 예외적인 상황이 있을까요?"**
 - 🐼 축 기준이 아니라 피처를 선형결합해서 대각선으로 자르는 Oblique Decision Tree, 또는 트리 결과를 로지스틱/신경망 같은 거리 기반 모델과 스택킹할 때.
- **Q2. "SVM 외에 스케일링이 필수적인 모델과 불필요한 모델을 2개씩 더 들어보세요."**
 - 🐼 필수: KNN, K-Means, PCA, 신경망, Ridge/Lasso. 불필요: XGBoost, LightGBM, CatBoost, Naive Bayes.
- **Q3. "규제 없는 순수 선형회귀는 스케일링이 필요한가요?"**
 - 🐼 공식으로 한 번에 풀면 계수만 단위에 맞춰 바뀔 뿐 예측은 같다. 하지만 경사하강으로 풀면 수렴 속도가 달라지고, 규제가 붙는 순간 답도 달라진다.

📖 용어를 말로 풀면:

- **단조 변환:** 원래 A가 B보다 컸다면 변환 후에도 A가 B보다 큰, 순서를 보존하는 변환. 100 곱하기, 로그 씌우기 등. 트리는 이 변환에 결과가 안 바뀐다.
- **마진:** 결정 경계(도로)와 가장 가까운 데이터(가장자리 건물) 사이의 도로 폭. SVM은 이 폭을 최대로 넓힌다.
- **스케일 불변성:** 단위(원 vs 만 원, cm vs m)가 바뀌어도 모델 판단이 안 바뀌는 성질.
- **조건수:** 손실 지형이 얼마나 길쭉한지. 클수록 경사하강이 지그재그로 느려진다.

2. 정규화(Min-Max)와 표준화(Z-score)의 차이 및 이상치 대응

질문: 정규화(Min-Max)와 표준화(Z-score)의 차이는 무엇이며, 이상치가 많은 데이터에서는 어떤 방식을 선택하며 그 이유는 무엇인가요?

🔗 반드시 입에 올려야 할 키워드:

- ❑ **정규화 (Min-Max):** 최솟값을 빼고 (최댓값 - 최솟값)으로 나눠 0~1 범위로 고정
- ❑ **표준화 (Z-score):** 평균을 빼고 표준편차로 나눠 평균 0, 표준편차 1로. 범위 제한 없음
- ❑ **이상치의 영향:** Min-Max는 최댓값에 이상치가 걸리면 정상 데이터 전부가 0 근처로 압축됨
- ❑ **이상치 많을 때의 선택:** 표준화는 완충일 뿐 → 중앙값과 IQR을 쓰는 **RobustScaler**, 또는 로그 변환·클리핑

💡 핵심 원리:

- **이상치가 Min-Max를 망치는 예시 (전교생 용돈):**
 - 100명의 용돈이 1만~5만 원 사이에 골고루 분포. 그런데 재벌 2세 한 명이 100억 원이라고 적었다.
 - Min-Max를 적용하면 재벌은 1.0, 나머지 99명은 0.000000~0.000004 사이. 즉 **99명이 사실상 전부 0**으로 뭉개져 변별력이 증발한다.
- **✅ 정정 — Z-score도 이 예시에서는 무력하다:**
 - 같은 데이터를 표준화하면 평균이 약 1억, 표준편차가 약 10억이 된다(100억 한 명이 둘 다 지배).
 - 정상 학생 99명은 전부 약 -0.1 근처, 서로 간 차이는 0.00004 수준. **Z-score도 정상 데이터를 한 점으로 뭉**갠다.
 - Min-Max보다 "덜 나뉘" 뿐이다(0~1에 갇히지 않고, 사람 수로 희석됨). 근본 해결이 아니다.
 - 따라서 "이상치가 많으면 표준화"라고만 답하면 부족. "**표준화는 완충, 진짜 답은 RobustScaler나 로그 변환, 클리핑**"으로 충위를 나눠 말한다.
- **RobustScaler (실무 답):**
 - 평균 대신 **100명 줄 세워 50번째 학생의 용돈(중앙값)**을 기준으로 빼고, **25번째와 75번째 학생의 용돈 차이 (IQR)**로 나눈다. 1등이 100억을 쓰든 1조를 쓰든 50번째 학생 값은 꿈쩍도 안 하므로 이상치에 방어된다.

🧠 심화 지식 (답변 재료):

- **둘 다 "직선 변환"이라는 것이 출발점:**
 - Min-Max든 Z-score든 "빼고 나누기"뿐이다. 그래서 **분포 모양(치우침, 봉우리 개수)은 못 바꾼다**. 위치와 크기만 바꾼다. "표준화하면 정규분포가 된다"는 오해 금지.
 - 모양을 바꾸려면 비선형 변환이 필요: 로그, Box-Cox, 분위수 변환(QuantileTransformer).
- **Breakdown point — "몇 명이 미쳐야 통계량이 망가지나":**
 - 최솟값/최댓값: **딱 1명**이 미친 값을 적으면 끝.
 - 평균/표준편차: 역시 1명으로 망가지지만, 사람 수로 희석되어 덜 흔들린다.
 - 중앙값: **절반이** 미쳐야 망가진다. IQR은 4분의 1. 그래서 "robust".
- **언제 Min-Max를 쓰는가 (반대편도 알아야 한다):**
 - 결과 범위가 유한해야 할 때: 이미지 픽셀(0~1), sigmoid/tanh 입력, 범위가 정해진 센서값. 이상치가 없고 범위를 아는 데이터.
 - Z-score: PCA, 선형모델, SVM, 대부분의 신경망 기본값.
- **이상치 도구 상자 (스케일러 외):**
 - 로그 변환(양수, 오른쪽 꼬리), winsorizing/클리핑(상하위 1%를 경계값으로 자름), 이상치 제거(IQR의 1.5배 규칙), 이상치 지시자 컬럼.
- **경사하강 수렴 속도의 기하 (꼬리질문 Q1 재료):**
 - 피쳐 A는 0~1, 피쳐 B는 0~100이면 손실 계곡이 B 방향으로 100배 가파르고 A 방향으로 완만한 **길쭉한 골짜기**가 된다.
 - 학습률은 **가장 가파른 방향에서 튀지 않도록** 작게 잡아야 한다. 그러면 완만한 방향은 거의 안 움직여서 수렴이 그만큼 느려진다.

- 이 "가장 가파름 대 가장 완만함"의 비율이 **조건수**. 표준화는 조건수를 1 근처로 만들어 둥근 그릇으로 편다.
- 덤: weight decay가 있으면 스케일이 피쳐별 규제 강도까지 바뀌어서 **최적해도** 달라진다.

🔍 꼬리 질문과 대응 포인트:

- **Q1. "딥러닝 경사하강법에서 표준화가 학습 속도를 올려주는 기하학적 이유는?"**
 - 🍌 타원형 계곡 비유. 스케일 차이가 크면 손실 지형이 길쭉한 V자 계곡이 되어 공이 양쪽 절벽 사이를 지그재그로 튕기며 느리게 내려간다. 표준화하면 둥근 밥그릇이 되어 최단 경로로 내려간다. (한 단어: 조건수가 작아진다.)
- **Q2. "표준화하면 데이터가 정규분포가 되나요?"**
 - 🍌 아니다. 직선 변환이라 모양은 그대로. 치우침을 없애려면 로그/Box-Cox/분위수 변환.
- **Q3. "RobustScaler를 쓰면 이상치는 어떻게 되나요?"**
 - 🍌 이상치는 여전히 매우 큰 값으로 남는다(제거가 아니다). 정상 데이터의 변별력을 지켜줄 뿐. 이상치 자체 처리는 클리핑/제거/지시자로 별도 결정.

📖 용어를 말로 풀면:

- **IQR (사분위수 범위):** 100명 줄 세웠을 때 25번째와 75번째 사이의 거리. 양 끝 극단값이 뭐든 가운데 50%는 영향 없음.
- **RobustScaler:** 평균 대신 중앙값, 표준편차 대신 IQR로 빼고 나누는 스케일러.
- **손실 등고선:** 손실이 같은 지점을 이은 선. 원형이면 최저점 찾기 쉽고, 길쭉하면 진동이 심해짐.
- **Breakdown point:** 통계량을 망가뜨리려면 데이터의 몇 %를 오염시켜야 하는가. 평균은 1개면 충분, 중앙값은 절반.

3. 결측치 대체 방식별 위험과 비무작위 결측(MNAR)의 문제점

질문: 결측치를 평균 대체, 중앙값 대체, 행 삭제, 모델 기반 대체로 처리하는 경우 각각 어떤 위험이 있는지, 결측이 무작위가 아닐 때 어떤 문제가 생기는지 설명해 주세요.

🎯 반드시 입에 올려야 할 키워드:

- ☐ **평균 대체:** 분산 축소, 이상치에 취약, 피쳐 간 상관관계 왜곡
- ☐ **중앙값 대체:** 이상치엔 강하나 분산 과소추정과 이분분포 왜곡은 동일
- ☐ **행 삭제:** 표본 급감(검정력 저하), 표본 편향
- ☐ **모델 기반 대체 (KNN/MICE):** 대체 모델의 과적합, 불확실성 무시, 계산 비용
- ☐ **MNAR (비무작위 결측):** 결측 자체가 의미를 가짐. 무작정 대체/삭제하면 대표성 훼손

💡 핵심 원리:

- **각 방식의 위험:**
 1. **평균 대체 (분산 축소):** 10점부터 90점까지 골고루 퍼진 100명 중 결측 40명을 전부 평균 50점으로 채우면, 데이터가 가운데로 뭉쳐 분산이 반토막. 모델은 "세상 사람 대부분은 50점"이라고 착각.
 2. **행 삭제 (표본 증발):** 결측이 하나라도 있는 행을 다 지우면 1만 건이 2천 건으로.
 3. **모델 기반 대체:** 결측을 맞히려고 또 다른 모델을 돌린다. 느리고, 대체 모델이 과적합하면 가짜 패턴이 원본에 영구 박제.
- **결측 메커니즘 3가지:**
 - **MCAR (완전 무작위):** 고양이에게 설문지에 커피를 쏟아 몇 줄이 지워짐. 결측 이유가 데이터와 무관 → 지워도 편향 없음.

- **MAR (조건부 무작위):** 여성 응답자가 체중란을 비율 확률이 높음. 결측 이유를 **다른 관측된 컬럼(성별)** 이 설명.
- **MNAR (비무작위, 가장 위험): 결측된 값 자체가 결측의 이유.**
 - 고소득자가 세무조사가 두려워 소득란을 비움. 중증 우울증 환자가 무기력해서 설문을 못 냄.
 - 이걸 평균으로 채우거나 삭제하면 "우리 병원엔 중증 환자가 한 명도 없다"는 왜곡된 결론.

🧠 심화 지식 (답변 재료):

- 세 메커니즘을 한 문장씩으로: MCAR은 결측이 **동전 던지기**. MAR은 결측 이유를 **다른 컬럼**이 설명. MNAR은 결측 이유가 **빈칸 그 자체**.
- 핵심 사실: MAR과 MNAR은 **데이터만으로 구분이 불가능하다**. 결측된 값에 달려 있는데 그 값이 없으니까. 도메인 지식으로 추론해야 한다. MCAR 여부만 통계 검정(Little's test)이 가능.
- 각 방법의 통계적 손상을 정확히:
 - **평균/중앙값 대체:** 결측 비율만큼 분산이 줄고(30% 결측이면 대략 분산 30% 감소), 다른 변수와의 **상관이 0 쪽으로 약해진다**(대체된 점들이 한 줄에 놓이니까). 표준오차가 작게 나와 유의성이 과대평가됨. 중앙값은 이상치엔 강하지만 이 문제는 그대로이고, 봉우리가 두 개인 분포는 그 사이 골짜기 값으로 채워 왜곡.
 - **행 삭제:** **MCAR일 때만 편향이 없다**. 표본 손실도 큼 — 컬럼 20개가 각각 5%씩 결측이면 살아남는 행은 약 36%뿐.
 - **회귀 대체:** 대체값이 회귀선 위에 정확히 찍혀 상관이 **과대**(평균 대체와 반대 방향 왜곡), 분산은 여전히 과소.
 - **KNN 대체:** 거리 기반이라 **스케일에 민감**(§1과 연결), 데이터 많으면 매우 느림.
 - **단일 대체의 공통 결함:** 대체값의 불확실성을 0으로 취급한다. → 다중 대체가 해답.
- **MICE가 좋은 진짜 이유 = 다중 대체:** 서로 다른 대체 데이터셋을 5~20개 만들고, 각각 분석한 뒤, 결과를 합칠 때 "데이터셋 안의 분산 + 데이터셋 사이의 분산"을 모두 더한다(Rubin's rules). "MICE가 정확하다"가 아니라 "**불확실성을 정직하게 보존한다**"가 포인트.
- 트리 계열은 결측을 그냥 먹는다: XGBoost/LightGBM은 NaN을 어느 가지로 보낼지 분할마다 학습 → 결측 지시자와 비슷한 효과를 자동으로 얻음.
- **MNAR 대응 도구 (지시자 외):** 통계 쪽에는 selection model, pattern-mixture model. 실무에선 **민감도 분석** — "결측값이 관측값보다 얼마나 다르다"는 가정을 바꿔가며 결론이 뒤집히는지 확인.
- **결측 대체도 누수 대상:** 대체에 쓰는 평균·중앙값·KNN 이웃은 **train에서만 계산** → §4와 연결.

🔍 꼬리 질문과 대응 포인트:

- **Q1. "MNAR이 의심될 때 가장 효과적인 전처리 트릭은?"**
 - 🐾 **결측 지시자 컬럼.** 소득은 중앙값으로 채워되, 옆에 "소득 미응답 여부" 컬럼을 만들어 1/0 기록. 그러면 모델이 "소득 숨긴 사람은 고소득자일 확률이 높다"는 패턴 자체를 학습.
- **Q2. "MAR인지 MNAR인지 데이터로 판별할 수 있나요?"**
 - 🐾 불가능. MNAR은 관측되지 않은 값에 달려 있어 데이터에 흔적이 없다. MCAR만 검정 가능. 나머지는 도메인 지식과 민감도 분석.
- **Q3. "평균 대체 대신 회귀 대체를 하면 상관관계 문제가 해결되나요?"**
 - 🐾 반대 방향으로 왜곡된다. 회귀선 위에 정확히 찍히니 상관이 과대. 노이즈를 더한 확률적 회귀 대체나 다중 대체가 필요.

📖 용어를 말로 풀면:

- **분산 과소추정:** 빈칸에 똑같은 값을 수십 개 복붙해서 데이터의 퍼짐이 가짜로 줄어드는 것.
- **표본 편향:** 특정 집단(고소득자, 중증 환자)만 빠져나가 남은 데이터가 전체를 대표하지 못하는 것.
- **MICE:** A가 비면 B, C를 보고 맞히고, B가 비면 A, C를 보고 맞히는 걸 핑퐁처럼 반복하는 대체법. 이를 여러 번 돌려 여러 데이터셋을 만드는 **다중 대체**가 본질.

- **Rubin's Rules:** 여러 대체 데이터셋의 결과를 합칠 때 "안의 분산 + 사이의 분산"을 모두 더해 불확실성을 정직하게 계산하는 규칙.

4. 전처리 통계량 계산 시점과 데이터 누수(Data Leakage)

질문: 전처리 통계량(평균, 표준편차 등)을 전체 데이터로 계산한 뒤 train/test를 나누면 어떤 문제가 발생하나요? 올바른 파이프라인은 어떻게 구성해야 할까요?

반드시 입에 올려야 할 키워드:

- ☐ **데이터 누수:** 테스트셋의 정보가 학습 단계로 흘러들어감
- ☐ **낙관적 성능 왜곡:** 검증 점수는 높게 나오지만 배포하면 폭락
- ☐ **올바른 순서:** 먼저 분할 → train으로만 fit → train과 test에 같은 통계량으로 transform
- ☐ **Pipeline:** 교차검증 시 각 fold 안에서 fit이 격리되도록 파이프라인 객체 사용

핵심 원리:

- **모의고사와 수능 비유:**
 - **잘못된 방식:** 수능 시험지의 단어 난이도 통계를 미리 훑쳐보고(전체 데이터로 통계 계산) 그에 맞춰 모의고사를 연습.
 - **결과:** 모의고사는 100점(낙관적 왜곡). 실제 수능장에서 새 지문을 만나면 폭락. 배포 후 강통 모델의 원인 1위.
- **fit과 transform의 역할:**
 - **fit** = 공식을 외우는 단계 (train에서 평균과 표준편차를 계산해 기억).
 - **transform** = 외운 공식을 적용하는 단계 (값에서 평균을 빼고 표준편차로 나눔).
 - **절대 규칙:** fit은 오직 train에서 한 번. test에는 새 통계를 배우면 안 되고, train에서 외운 공식으로 transform만.
- **올바른 5단계:**
 1. 가장 먼저 train/test를 물리적으로 쪼갬다.
 2. scaler를 train에만 fit.
 3. train을 transform.
 4. test를 transform (**test에 fit을 절대 부르지 않는다**).
 5. 교차검증 때는 밖에서 스케일링하지 말고 Pipeline(스케일러 + 모델)을 만들어 fold마다 알아서 격리되게 한다.

심화 지식 (답변 재료):

- **누수를 3층위로 구분하면 답이 깊어진다:**
 1. **전처리 누수:** 스케일러/대체기/인코더를 전체로 fit. 영향은 작지만 0이 아님(특히 소표본, 이상치 있을 때).
 2. **타겟 누수:** 타겟이 결정된 **이후** 생기는 정보가 피처에 있음. 예: "항생제 처방 여부"로 감염 예측, "환불 여부"로 사기 예측. 전체 데이터 기반 타겟 인코딩·피처 선택도 여기 속함.
 3. **분할 누수:** 같은 환자/사용자가 train과 test에 모두 있음(→ 그룹 단위 분할), 시계열을 랜덤 분할(→ 시간순 분할), 중복 행.
- **ESL 교과서의 "잘못된 교차검증" 예시 (외워두면 강력):** 라벨과 완전히 무관한 랜덤 데이터에서, 전체 데이터로 상관 높은 피처 100개를 먼저 고른 뒤 교차검증하면 오차가 거의 0으로 나온다. 피처 선택이 fold 밖에서 이뤄졌기 때문. → **"모든 학습 개입(선택, 인코딩, 대체, 스케일, 오버샘플링)은 fold 안에서"** 가 원칙.
- **왜 스케일러 누수는 "덜" 위험하고 타겟 누수는 치명적인가:**
 - 스케일러 통계는 입력값의 **비지도** 통계이고 train/test 분포가 비슷하면 거의 같은 값 → 평가가 조금만 낙관적.

- 타겟 인코딩은 **정답의 통계가 입력으로** 흘러간다. 특히 회소 카테고리는 자기 자신의 정답을 그대로 피쳐로 갖게 됨 → train 오차가 인위적으로 0에 가까워짐.
- 올바른 타겟 인코딩: **out-of-fold** — 각 행의 인코딩 값을 "그 행이 속하지 않은 다른 fold"의 통계로 계산. 더하기 스무딩.
- **오버샘플링(SMOTE)도 fold 안에서:** 전체에 SMOTE 후 분할하면 합성 샘플의 원본이 test에 남아 근접 중복 누수. sklearn Pipeline은 샘플 수를 바꾸는 단계를 지원하지 않으므로 imblearn의 Pipeline 사용.
- **배포 관점:** 학습 때 fit한 scaler/encoder 객체를 모델과 **함께 저장**해야 서빙에서 같은 통계 사용. 서빙 데이터로 다시 fit하면 train-serving skew.
- **누수 판별 한 문장 (Kaufman 2012):** "예측 시점에 이 값을 정당하게 알 수 있는가?" 아니면 누수.

🔍 꼬리 질문과 대응 포인트:

- **Q1. "타겟 인코딩이나 피쳐 선택 누수가 스케일러 누수보다 훨씬 위험한 이유는?"**
 - 🐼 스케일러 누수는 입력값 크기 정보가 새는 것이지만, 타겟 인코딩 누수는 **맞혀야 할 정답의 통계가 입력에 직접 섞이는 것**. 정답지를 보고 푸는 꼴이라 train 오차가 0에 가까워지고 배포하면 망가진다.
- **Q2. "시계열 데이터에서 랜덤 분할을 하면 어떤 문제가 있나요?"**
 - 🐼 미래 시점 샘플이 train에 섞여 과거를 예측하는 데 쓰인다(시간 누수). 시간순 분할(TimeSeriesSplit). 이동평균 같은 피쳐도 미래 값을 포함하면 안 된다.
- **Q3. "Pipeline을 쓰면 왜 누수가 자동으로 방지되나요?"**
 - 🐼 교차검증 함수가 fold마다 파이프라인 전체를 복제해 그 fold의 train으로만 fit하고 검증 fold에는 transform만 적용한다. 전처리와 모델이 한 객체라 순서를 실수할 여지가 없다.

📖 용어를 말로 풀면:

- **데이터 누수:** 학습에 들어가면 안 될 미래 정보나 시험 정보가 흘러들어가 모델이 부정행위를 하는 것.
- **Pipeline:** 전처리와 모델을 기차 칸처럼 연결해, 교차검증이나 새 데이터 입력 시 누수 없이 순서대로 처리하는 도구.
- **Out-of-fold 인코딩:** 각 행의 타겟 인코딩 값을 자기가 속하지 않은 fold의 통계로 계산해서, 자기 정답을 자기 피쳐로 못 쓰게 막는 것.
- **Train-serving skew:** 학습 때와 서빙 때 전처리가 미묘하게 달라 배포 후 성능이 떨어지는 현상.

5. 데이터 증강의 일반화 원리 (정규화와 불변성)

질문: 데이터 증강이 일반화 성능을 높이는 원리를 정규화(regularization)와 불변성(invariance) 관점에서 설명해주세요.

🌀 반드시 입에 올려야 할 키워드:

- ☐ **불변성:** 입력을 기하학적·색상적으로 변형해도 본질(클래스)은 유지됨을 학습
- ☐ **정규화 효과:** 데이터 다양성을 넓혀 픽셀 위치·노이즈 패턴 암기(과적합)를 방지
- ☐ **결정 경계 평활화:** 희소한 공간을 가상 데이터로 채워 부드럽고 강건한 경계
- ☐ **사전 지식 주입:** "사물은 회전해도 같은 사물"이라는 상식을 데이터 형태로 전달

💡 핵심 원리:

- **불변성 관점 (친구 얼굴 알아보기):**
 - 친구가 안경을 쓰든, 고개를 갸우뚱하든, 어두운 조명이든 우리는 그 친구임을 안다. 뇌가 회전·이동·조명 변화에 흔들리지 않는 불변성을 갖기 때문.
 - 신경망은 픽셀 단위 숫자를 보므로 고양이 사진을 1픽셀만 밀어도 다른 데이터로 볼 수 있다. 증강은 "살짝 돌리거나 색이 바뀌어도 본질은 그대로"라는 대칭성을 가르친다.

- **정규화 관점 (죽보 외우는 학생 참고육):**

- 수역 파라미터의 모델은 엄청난 암기왕. 1만 장을 주면 "좌측 상단에 파란 픽셀, 3번째 줄에 흰 점이면 고양이"처럼 픽셀 위치를 통째로 암기한다.
- 증강을 걸면 때 에포크 이미지가 조금씩 잘리고 돌아가서 **100번을 돌아도 완전히 같은 이미지가 한 번도 안 들어온다.**
- 꿈수 암기가 불가능해지고 물체의 본질적 실루엣을 강제로 배운다. 이것이 Dropout, Weight Decay와 같은 정규화 효과.

 심화 지식 (답변 재료):

- **정규화 관점의 이론적 뿌리 3개 (하나만 말해도 차별화):**

1. **Bishop(1995) — 노이즈 학습은 규제와 같다:** 입력에 작은 노이즈를 섞어 학습하면 "입력이 조금 바뀌어도 출력이 크게 안 바뀌는" 모델이 되도록 강제된다. 이것이 수학적으로 L2 규제와 같은 효과임이 증명되어 있다. 한 문장으로: "살짝 흔들어도 답이 안 바뀌게 만들어라"를 벌금 대신 데이터로 강제하는 것.
2. **VRM (근방 위험 최소화):** 보통 학습은 "내가 본 점 하나만 정답"이라고 믿는다. VRM은 "그 점 근처도 다 같은 정답"이라고 믿고 근처에서 샘플을 뽑아 학습한다. 증강이 바로 그 "근처에서 뽑기". 빈 공간이 채워지니 결정 경계가 관측점에 딱 붙지 않는다. Mixup 논문이 이 틀을 쓴다.
3. **편향-분산 관점:** 유효 데이터 수가 늘어 분산이 줄고, 동시에 "변환에 불변인 함수"로 가설 공간이 제한되어 유효 용량이 줄어든다 → 과적합 여지 축소.

- **불변성 관점 — 구조가 못 넣는 사전지식을 데이터로 넣는다:**

- CNN 구조가 내장한 것은 평행이동에 대한 등변성뿐(합성곱의 가중치 공유). 회전·스케일·조명·시점 불변성은 구조에 없다.
- 증강은 모델이 "변환된 입력에도 같은 출력"을 내도록 강제 → 픽셀 좌표가 아닌 형태 특징에 의존하게 됨.
- 구조 자체에 불변성을 넣는 연구도 있지만, 증강이 훨씬 범용적이고 싸다.
- **불변성 vs 등변성:** 불변 = 입력이 변해도 출력 동일(분류). 등변 = 입력이 변하면 출력도 같은 방식으로 변환(세그멘테이션, 검출 — 박스도 같이 움직여야).

- **Mixup / CutMix:**

- Mixup: 두 이미지를 비율대로 섞고(예: 고양이 70% + 강아지 30%), 라벨도 같은 비율로 섞는다. 샘플 **사이**에서 선형으로 동작하도록 유도.
- CutMix: 이미지 일부를 다른 이미지 패치로 교체, 라벨은 **면적 비율**. Mixup보다 자연스러운 이미지(국소 특징 보존), 검출·지역화에 유리.
- 공통 효과: 소프트 라벨 → 과신 억제(캘리브레이션 개선), 라벨 스무딩과 유사, 적대적 예제에 강건, 클래스 사이 경계 완화.

- **정규화 도구들의 관계:** Dropout/Weight decay는 **모델 파라미터**에 작용, 증강은 **데이터**에 작용 → 상호보완적이라 함께 쓴다. Early stopping은 **학습 시간**에 작용.
- **트레이드오프:** 과도한 증강은 train 분포가 test 분포에서 멀어져 **underfitting** 또는 분포 이동. 증강 강도도 하이퍼파라미터(RandAugment는 강도와 개수를 튜닝). 학습 후반에 증강을 줄이는 기법도 있다.
- **표 형식 데이터:** SMOTE(소수 클래스 보간), 노이즈 주입, 피쳐 드롭 정도. 이미지처럼 자명한 불변성이 없어 효과가 제한적 → "증강은 도메인 지식이 있어야 설계 가능"이라는 말로 연결.

 꼬리 질문과 대응 포인트:

- **Q1. "Mixup이나 CutMix는 고전적 회전/이동 증강과 정규화 관점에서 어떻게 다른가요?"**

- 🐾 고전 증강은 "고양이를 돌려도 100% 고양이"라는 클래스 내부 불변성을 가르친다. Mixup/CutMix는 고양이와 강아지를 섞고 라벨도 섞어서, 모델이 "무조건 100% 고양이"라고 과신하는 것을 막고 클래스 사이 경계를 부드럽게 만든다.

- **Q2. "증강이 Weight Decay와 같은 정규화라면 그냥 Weight Decay를 세게 걸면 안 되나요?"**

- 🐼 작용 대상이 다르다. Weight decay는 파라미터 크기를 줄일 뿐 "회전해도 같은 물체"라는 **도메인 지식**은 담지 못한다. 그건 증강만 할 수 있고, 둘은 상호보완적.
- **Q3. "증강을 너무 세게 하면?"**
 - 🐼 학습 분포가 실제와 멀어져 underfitting. 증강 강도도 튜닝 대상.

📖 용어를 말로 풀면:

- **불변성**: 그림을 뒤집거나 비틀어도 정답이 안 흔들리는 성질.
- **등변성**: 입력을 움직이면 출력도 같은 방식으로 움직이는 성질. 합성곱이 평행이동에 대해 갖는 성질.
- **결정 경계 평활화**: 경계를 칼로 벤 듯 뾰족하게 그리지 않고 둥글게 그려 새 데이터에도 오판하지 않게 하는 것.
- **VRM**: "본 점 하나만 믿지 말고 그 주변도 같은 정답일 것"이라 가정하고 학습하는 틀. 증강의 이론적 언어.

6. 숫자 인식과 레이블 보존(Label-preserving) 증강 기준

질문: 숫자 인식 데이터에 좌우 반전이나 180도 회전을 적용하면 어떤 문제가 생길까요? 레이블 보존 관점에서 증강 기법을 선택하는 기준을 말씀해 주세요.

🎯 반드시 입에 올려야 할 키워드:

- ☐ **레이블 보존**: 변환 후에도 원래 정답의 의미가 100% 유지되어야 함
- ☐ **의미 왜곡 사례**: 180도 회전 시 6과 9가 뒤바뀜, 좌우 반전 시 2·3·5·7이 거울 문자가 됨
- ☐ **선택 기준**: 도메인 지식으로 클래스 본질이 안 깨지는 변환만(미세 회전 $\pm 10^\circ$, 미세 이동, 노이즈, 탄성 왜곡 등)

💡 핵심 원리:

- **황금률**: "변형된 이미지를 사람이 봐도 원래 정답과 일치해야 한다."
- **숫자에 반전/회전을 걸면 벌어지는 일**:
 - **180도 회전**: 6을 돌리면 완벽한 9. 이걸 정답 6이라고 주입하면 모델은 9를 보고 6이라 부른다. (자율주행차가 60km 표지판을 90km로 읽는 사고)
 - **좌우 반전**: 3을 뒤집으면 현실에 없는 외계 문자. 쓰레기 데이터를 학습.
- **도메인별 가이드**:
 - 일반 사진(강아지, 자동차): 좌우 반전 OK.
 - 문자/숫자: 좌우 반전 절대 불가. 대신 $\pm 10^\circ$ 이내 미세 회전, 획 두께 늘리기/줄이기, 1~2픽셀 밀기만.

🧠 심화 지식 (답변 재료):

- **판단 원칙을 두 조건으로**:
 1. 허용하는 변환은 "**라벨이 변하지 않는 변환**"이어야 한다.
 2. 동시에 그 변환은 **테스트/배포 시 실제로 발생하는 변형**을 닮아야 한다(그래야 배운 불변성이 쓸모 있다). 손글씨는 $\pm 15^\circ$ 기울기, 획 두께, 위치 흔들림이 실제 변동. 현실에서 안 일어나는 변형(숫자 좌우 반전)은 라벨을 안 깨더라도 낭비.
- **문자에서 깨지는 사례 (즉답용)**:
 - 180도 회전: 6과 9. 90도 회전: N과 Z. 좌우 반전: b와 d, p와 q, 2·3·5·7. 상하 반전: b와 p, d와 q. 0·1·8은 거의 대칭이라 무해.
- **손글씨에 안전한 증강**: 소각도 회전, 기울임(shear), 미세 이동/스케일, **탄성 왜곡**(Simard 2003 — 고무판처럼 국소적으로 늘였다 줄여 손 떨림을 모사, MNIST 오차를 크게 낮춘 고전 기법), 획 두께 팽창/침식, 소량 노이즈, 일부 가림.
- **도메인별 금지 목록 (한 줄씩)**:
 - 일반 사물: 상하 반전은 보통 NO (뒤집힌 자동차는 드뭄).
 - 흉부 X-ray: 좌우 반전 주의 — 심장 위치(우심중) 정보가 깨짐. 실무에서 쓰는 경우도 있어 "주의"로 답변.

- 도로 표지판: 좌우 반전 금지 — 좌회전이 우회전이 됨.
- 음성 화자 인식: 피치 시프트 금지(화자 정체성이 피치에). 음성 인식(내용)은 피치 시프트 OK.
- 시계열: 순서 섞기 금지.
- **역발상 — 라벨을 함께 바꾸는 증강:** 임의 방향에서 찍힌 숫자를 인식해야 한다면, 6을 180도 돌리고 **라벨을 9로 바꾸**는 증강을 설계할 수 있다. "레이블 보존"은 라벨을 고정한다는 뜻이 아니라 **"이미지와 라벨의 일관성 유지"**라는 뜻. 이 관점을 말하면 이해도가 드러난다.
- **검출/세그멘테이션에서는 라벨도 변환:** 좌우 반전 시 박스 좌표와 마스크도 같이 반전(등변성). 라벨 보존 = "라벨을 같은 변환으로 따라 바꾼다"로 확장.

🔍 꼬리 질문과 대응 포인트:

- **Q1. "의료 영상이나 도로 표지판에서 좌우 반전을 함부로 쓰면 안 되는 이유는?"**
 - 🐼 흉부 X-ray는 심장 위치 등 장기의 해부학적 위치가 진단에 중요해서 뒤집으면 우심증 등 오진. 표지판은 좌회전/우회전 화살표가 반대로.
- **Q2. "임의 각도로 회전된 숫자를 인식해야 한다면?"**
 - 🐼 회전 시 라벨이 바뀌는 쌍(6/9)만 라벨을 함께 바꾸는 증강, 방향을 먼저 정규화하는 전처리, 또는 회전 각도를 보조 라벨로 함께 예측.
- **Q3. "레이블 보존 여부를 실무에서 어떻게 검증하나요?"**
 - 🐼 증강 샘플을 무작위로 뽑아 사람이 라벨링해 원본과 일치율 확인. 증강 추가 후 검증 성능이 떨어지면 라벨 파괴를 의심.

📖 용어를 말로 풀면:

- **레이블 보존 변환:** 데이터를 찌그러뜨리거나 돌려도 정답이 안 망가지는 안전한 변형.
- **의미론적 역전:** 변형했더니 아예 다른 클래스로 뒤집히는 치명적 오류 (6→9, 좌회전→우회전).
- **탄성 왜곡:** 이미지를 고무판처럼 국소적으로 늘였다 줄이는 변형. 손글씨 떨림을 흉내 내 MNIST에서 가장 효과적인 증강 중 하나.

7. 검증/테스트 데이터 증강 적용 여부 및 TTA (Test-Time Augmentation)

질문: 증강을 검증/테스트 데이터에도 적용해야 할까요? 테스트 시 증강은 어떤 경우에 의미가 있을까요?

🎯 반드시 입에 올려야 할 키워드:

- ☐ **기본 원칙:** 검증/테스트에는 **무작위 증강**을 쓰지 않는다. 단, 크기 조정·중앙 자르기·정규화 같은 **결정적 전처리**는 **train과 동일하게 적용**
- ☐ **이유:** 배포 환경 분포를 그대로 모사하고, 재현 가능하게 평가하기 위해
- ☐ **예외 (TTA):** 테스트 이미지 1장에 여러 변환을 걸어 N개 사본 생성
- ☐ **TTA 작동:** N개 예측 확률을 평균(soft voting)해 분산을 줄임 — 앙상블 효과
- ☐ **TTA 비용:** 추론 시간과 연산이 N배

💡 핵심 원리:

- ☒ **정정 — "증강 없음"의 정확한 의미:**
 - 끄는 것은 **무작위 증강**(랜덤 크롭, 랜덤 반전, 색 지터)이다.
 - 크기 조정, 중앙 자르기, 텐서 변환, 평균/표준편차 정규화는 **train과 똑같이** 적용해야 한다. 이걸 빼면 입력 분포가 달라져 성능이 폭락한다.
 - 전형적 예: train은 "랜덤 크롭 + 랜덤 반전 + 정규화", val/test는 "리사이즈 + 중앙 크롭 + 정규화".

- 무작위 증강을 쓰면 안 되는 이유 (수능 시험지): 모든 수험생이 똑같이 깨끗한 문제지를 받아야 공정 비교. 테스트셋을 막 돌리고 노이즈를 뿌리면 점수가 들쭉날쭉해 모델 A가 B보다 나은지 판단 불가.
- TTA (의사의 다각도 진찰):
 - X-ray가 애매하면 의사는 확대도 해보고, 밝기도 올리고, 살짝 돌려보며 여러 번 본 뒤 종합 판정.
 - 작동: 이미지 1장 → [원본, 좌우반전, 약간 확대, 밝기 조절] 4개 사본 → 모델 4번 통과 → 확률(80%, 75%, 85%, 70%)을 평균해 77.5%로 결론.
 - 효과: 예측 흔들림이 줄어 캐글·의료에서 1~2% 상승.
 - 단점: 연산과 응답 시간이 4~5배.

🧠 심화 지식 (답변 재료):

- 검증셋이 재현 가능해야 하는 진짜 이유: 검증셋은 **early stopping**과 **모델/하이퍼파라미터 선택의 기준**이다. 무작위 증강이 들어가면 같은 모델을 두 번 평가해도 점수가 달라 "에포크 12가 15보다 좋다"는 판단 자체가 노이즈가 된다.
- 일부러 테스트를 손상시키는 경우 (구분해서 말하기): ImageNet-C처럼 노이즈·블러·날씨 손상을 가한 테스트셋으로 강건성을 재는 벤치마크가 있다. 이는 정확도가 아닌 강건성이라는 **별도 지표** — 표준 평가와 혼동하지 않기.
- TTA의 역사: AlexNet(2012)이 **10-crop**(네 모서리 + 중앙, 각각 좌우 반전) 평균으로 오차를 낮췄다. VGG는 여러 스케일로 평가. 즉 TTA는 최신 트릭이 아니라 딥러닝 초창기부터의 표준.
- TTA가 작동하는 이유: 입력 공간에서의 **양상불**. 변환본마다 예측이 조금씩 다르고 서로 완전히 같이 움직이지 않으므로 평균하면 흔들림이 줄어든다. 조건: 모델이 그 변환에 **준불변**이어야 함 → **학습 때 쓴 변환과 같은 변환**을 써야 효과. 학습에 없던 변환을 TTA에 쓰면 오히려 악화.
- 세그멘테이션/검출에서의 TTA: 반전해서 예측한 뒤 출력을 **다시 반전**해 원래 좌표계로 돌린 후 평균. 검출은 박스 병합(WBF, NMS)도 필요.
- 부가 활용: 변환본 간 예측 **편차**로 불확실성 추정(예측이 흔들리면 신뢰도 낮음).
- 비용 계산: N개 사본을 한 배치로 묶으면 지연은 N배보다 덜 늘지만 연산량은 정확히 N배. 실시간 예산(초당 30프레임 = 33밀리초)에서는 불가. 오프라인 배치 추론, 의료 판독, 캐글에 적합. 일반적 이득 0.5~2%p.
- 대안: TTA 대신 모델 앙상블, 또는 지식 증류로 TTA 효과를 단일 모델에 흡수.

🔍 꼬리 질문과 대응 포인트:

- Q1. "TTA를 실시간 서비스(자율주행, 실시간 탐지)에 쓰기 어려운 이유는?"
 - 🖱️ 초당 30프레임을 33밀리초 안에 처리해야 하는데 TTA는 4~5배 느려져 사고로 이어진다. 1~2초 지연이 허용되는 의료 판독이나 오프라인 대량 분석에 국한.
- Q2. "검증셋에 정규화도 빼야 하나요?"
 - 🖱️ 아니다. 정규화/리사이즈/중앙 크롭은 결정적 전처리이며 train과 동일하게 적용. 빼는 것은 무작위 증강뿐.
- Q3. "TTA에 어떤 변환을 골라야 하나요?"
 - 🖱️ 학습 때 쓴 라벨 보존 변환 중 결정적으로 열거 가능한 것(좌우 반전, 몇 개의 스케일/크롭). 학습에 없던 변환은 모델이 불변하지 않아 역효과.

📖 용어를 말로 풀면:

- TTA: 시험 볼 때 문제 사진을 다각도로 비틀어 여러 번 풀고 평균 내는 추론 전용 앙상블.
- 소프트 보팅: 여러 예측의 확률값을 평균해 최종 답을 정하는 부드러운 다수결.
- 결정적 전처리 vs 무작위 증강: 같은 입력에 항상 같은 출력(리사이즈, 정규화)은 전처리, 매번 다른 결과(랜덤 크롭, 반전)는 증강. 평가 시엔 전자만 남긴다.

8. Batch GD, SGD, Mini-batch GD의 비교 (분산과 계산 비용)

질문: Batch GD, SGD, Mini-batch GD의 차이를 그래디언트 추정의 분산과 계산 비용 측면에서 비교해 주세요.

🌀 반드시 입에 올려야 할 키워드:

- ❑ **Batch GD**: 전체 데이터로 1회 업데이트 → 분산 0, 메모리/시간 비용 최대, 안장점 탈출 어려움
- ❑ **SGD**: 샘플 1개마다 업데이트 → 분산 최대(심한 진동), 비용 최소, 노이즈로 로컬 미니마 탈출 가능하나 수렴 불안정, GPU 병렬 활용 불가
- ❑ **Mini-batch GD**: N개 묶음 → 적절한 분산과 안정성, GPU 병렬 처리 극대화, 현대 표준

💡 핵심 원리 (등산객 하산 비유):

- 안개 낀 밤 산꼭대기에서 가장 낮은 골짜기로 내려가는 등산객:
 - 1. Batch GD (전체 지도 보고 한 걸음)**: 산 전체 지도를 1시간 펼쳐보고 완벽한 방향으로 딱 한 걸음. *문제*: 데이터가 수백만이면 지도가 배낭(메모리)에 안 들어감. 평지(안장점)를 만나면 기울기 0이라 멈춤.
 - 2. SGD (발밑 돌맹이 하나 차보고 한 걸음)**: 눈 감고 돌맹이 1개만 차본 뒤 그 경사로 달림. *문제*: 빠르고 요동쳐서 평지는 탈출하지만 지그재그로 비틀거려 골짜기 중심에 안착 못 함. GPU 수천 코어에 1개씩 던지니 하드웨어가 논다.
 - 3. Mini-batch (주변 32~64개 지형 보고 한 걸음)**: 주변 평균 경사를 나침반 삼아 안정적으로 달림. *장점*: GPU 메모리에 맞고, 수천 코어가 한 번에 병렬 연산해 극도로 빠름. 적당한 흔들림으로 함정도 탈출.
- Sharp Minima vs Flat Minima**:
 - Sharp (*바늘구멍*): 훈련 오차 0이지만 테스트 데이터로 1mm만 어긋나도 튕겨 나가 오차 폭발(과적합). 큰 배치가 잘 빠진다.
 - Flat (*넓은 대야*): 위치가 조금 어긋나도 오차가 거의 안 늘어 안정적(일반화 우수). 적당한 미니배치 노이즈가 파라미터를 튕겨 Flat으로 안내.

🧠 심화 지식 (답변 재료):

- 말로 잡아두는 핵심 관계**:
 - 미니배치로 계산한 기울기는 "평균적으로는 맞지만 매번 흔들리는" 추정치. 흔들림(분산)은 **배치 크기에 반비례**, 한 스텝 비용은 **배치 크기에 비례**.
 - 1 에포크의 총 계산량은 방식과 무관하게 같다(데이터 전체를 한 번 훑으니). 차이는 **업데이트 횟수** — SGD는 데이터 수만큼, BGD는 1번. 그래서 "에포크당"이 아니라 "스텝당"으로 비교해야 한다.
- 수렴 이야기 한 줄**: BGD는 진짜 기울기를 따라가니 최적점에 정확히 도착. SGD는 학습률을 고정하면 최적점 근처에서 계속 흔들리며 못 앓는다 → 앓으려면 **학습률을 점점 줄이는 스케줄**이 필요. 너무 빨리 줄이면 도착 전에 멈추고, 안 줄이면 계속 떨린다.
- 하드웨어 관점**:
 - GPU는 배치 차원을 행렬곱으로 병렬화. 배치 1은 코어 대부분이 유휴. 배치를 키울수록 처리량 증가.
 - 그러나 **critical batch size** (OpenAI 2018) 이후엔 배치를 더 키워도 스텝당 얻는 정보가 포화되어 필요한 스텝 수가 안 줄어듦 → 연산 낭비.
 - 메모리는 배치 크기에 비례. BatchNorm은 배치가 16 이하면 배치 통계가 불안정(→ GroupNorm/LayerNorm 대안).
 - 관례: 32~256, 2의 거듭제곱(메모리 정렬·커널 효율).
- 일반화 — 양쪽 주장을 모두 알아야 한다**:
 - Keskar(2016): 대배치 → sharp minima → 일반화 갭. 노이즈 크기는 대략 "학습률 나누기 배치 크기"에 비례하는 "온도"로 작용해 좁은 골짜기에서 튕겨 나오게 한다.
 - 반론: Goyal(2017) **Linear Scaling Rule** — 배치를 k배 키우면 학습률도 k배, 초반엔 warmup. 이렇게 ImageNet을 배치 8192로 정확도 손실 없이 학습. Hoffer(2017) "Train longer, generalize better" — 갭은 상당 부분 스텝 수 부족 때문.
 - "대배치 = 무조건 나쁨"이 아니라 "**학습률·스텝 수 조정 없이** 키우면 나쁨"이 정확한 표현.

- **안장점:** 고차원에서는 로컬 미니마보다 안장점이 압도적으로 많다(Dauphin 2014). BGD는 안장점 근처에서 기울기가 거의 0이라 극도로 느려지고(엄밀히 "영원히 멈춤"은 아님), SGD 노이즈가 탈출시킨다.
- **셔플링:** 매 에포크 순서를 섞어야 미니배치가 편향되지 않는다. 정렬된 데이터로 배치를 만들면 클래스가 배치마다 몰려 기울기가 편향된다.

🔍 꼬리 질문과 대응 포인트:

- **Q1. "미니배치를 극단적으로 크게(예: 16,384) 키우면 왜 일반화가 떨어지나요?"**
 - 🐼 노이즈가 사라져 BGD처럼 동작 → sharp minima에 갇히기 쉬움. 적당한 노이즈가 정규화 역할을 해 flat minima를 찾게 돕는다. (덧붙이면: 학습률 선형 스케일링 + warmup + 충분한 스텝으로 갭을 상당히 줄일 수 있다.)
- **Q2. "배치를 2배로 키우면 학습률은?"**
 - 🐼 Linear scaling rule — 학습률도 2배, 초반 warmup 필수. 근거: 기울기 흔들림이 절반이라 더 큰 걸음을 믿을 수 있음. Adam 계열은 제공된 배를 권하는 연구도 있음.
- **Q3. "SGD는 매 스텝 노이즈가 있는데 어떻게 수렴하나요?"**
 - 🐼 평균적으로는 옳은 방향(불편 추정). 다만 학습률을 고정하면 근처에서 진동하므로 학습률을 점차 줄이는 스케줄(step, cosine)로 수렴.

📖 용어를 말로 풀면:

- **그레디언트 분산:** 일부 데이터로 계산한 기울기가 전체 데이터의 진짜 기울기에 비해 얼마나 흔들리는가. 배치 크기에 반비례.
- **안장점:** 말 안장처럼 앞뒤로는 오르막, 좌우로는 내리막이라 기울기가 0이 되어 경사하강이 최저점인 줄 알고 멈추는 지형.
- **Flat vs Sharp Minima:** 넓은 대야형 골짜기(새 데이터에 강건) vs 바늘구멍형(새 데이터에 무너짐).
- **Linear Scaling Rule:** 배치를 k배 키우면 학습률도 k배, 초반엔 서서히 올리는(warmup) 규칙.
- **Critical Batch Size:** 여기까지는 배치를 키운 만큼 스텝 수가 줄지만, 넘어가면 배치를 키워도 스텝 수가 안 줄어 연산만 낭비되는 경계선.

9. Adam의 1차/2차 모멘트와 편향 보정(Bias Correction)

질문: Adam의 1차 모멘트와 2차 모멘트는 각각 어떤 역할을 하며, 편향 보정은 왜 필요한가요?

🎯 반드시 입에 올려야 할 키워드:

- ☐ **1차 모멘트 (m, 모멘텀):** 기울기의 지수이동평균 → 방향에 관성 부여, 진동 완화·가속
- ☐ **2차 모멘트 (v, RMSProp):** 기울기 제곱의 지수이동평균 → 파라미터별 적응형 학습률
- ☐ **편향 보정 필요 이유:** m과 v를 0에서 시작하므로 초반에 실제보다 작게 추정됨
- ☐ **보정 방법:** "1에서 감쇠율의 스텝 수 제곱을 뺀 값"으로 나눔. 초반엔 이 값이 1보다 작아서 나누면 원래 크기로 복원, 스텝이 쌓이면 1에 가까워져 보정이 자연스럽게 사라짐
- ☐ **보정 없을 때의 실제 증상** ✅ **정정:** v가 m보다 훨씬 심하게 축소되어 그 비율이 커짐 → 초반 스텝이 지나치게 커져 불안정/발산

💡 핵심 원리 (눈썰매와 자동 브레이크):

- Adam은 두 장치를 합친 것:
 1. **1차 모멘트 = 무거운 눈썰매의 관성:** 작은 요철이나 평지가 나와도 관성으로 뚫고 지나간다. 지그재그를 상쇄하고 내리막에 가속.

2. **2차 모멘트 = 똑똑한 자동 브레이크**: 가파른 절벽(기울기 큰 방향)에서는 보폭을 좁히고, 완만한 평지(기울기 작은 방향)에서는 보폭을 넓힌다.

• **편향 보정 (체중계 비유):**

- 매일 "어제 90% + 오늘 10%"로 몸무게 이동평균을 낸다. 시작값은 기록이 없으니 0kg으로 세팅.
- 첫날 실제 70kg → 계산은 0의 90% + 70의 10% = **7kg**. 실제 70인데 7로 인식.
- 2차 모멘트는 감쇠율이 0.999라 오늘 값의 0.1%만 반영 → **1000배 축소**.
- 보정: 첫날은 "1 빼기 0.9 = 0.1"로 나눈다 → 7 나누기 0.1 = **70kg** 복원. 2차 모멘트는 0.001로 나눠 복원.

• **✓ 정정 — 보정이 없으면 "학습이 멈추는" 게 아니라 "스텝이 폭주"한다:**

- m은 10배, v는 1000배 축소된다. 그런데 실제 스텝은 "m을 v의 제곱근으로 나눈 것".
- m은 10분의 1, v의 제곱근은 약 32분의 1 → 비율이 **약 3배 커진다** → 첫 스텝이 의도의 3배.
- 두 모멘트가 **불균등하게** 축소되어 비율이 커지고, 초반에 파라미터가 크게 튀어 불안정하거나 발산. Adam 논문도 "보정 없는 RMSProp은 초반 스텝이 매우 커져 자주 발산"이라고 명시.
- 면접에서 "값이 0이 되어 학습이 안 된다"고 하면 방향이 틀린 것.

🧠 **심화 지식 (답변 재료):**

• **갱신 절차를 말로 (반드시 외울 것):**

- 1차 모멘트 $m = \text{이전 } m \text{의 } 90\% + \text{이번 기울기의 } 10\%$. (방향·관성)
 - 2차 모멘트 $v = \text{이전 } v \text{의 } 99.9\% + \text{이번 기울기 제곱의 } 0.1\%$. (크기)
 - 둘 다 편향 보정: "1 빼기 감쇠율의 스텝수 제곱"으로 나눈다.
 - 파라미터 갱신 = 학습률 $\times$ (보정된 m) $\div$ (보정된 v 의 제곱근 + 아주 작은 수).
- 기본값: 학습률 0.001, 감쇠율 0.9와 0.999, 아주 작은 수는 10의 -8제곱.

• **편향 보정이 왜 하필 그 값으로 나누는가:** 0에서 시작한 이동평균은 t번째 시점에 "진짜 값의 (1 - 감쇠율의 t제곱) 배"만 채워져 있다. 1일째는 10%만, 2일째는 19%만... 그러니 딱 그 비율로 나눠주면 원래 크기가 된다. 수학적으로 정확히 유도된 보정이지 "뺑튀기"가 아니다.

• **Adam의 성질 (꼬리질문용):**

- **유효 스텝 크기 $\approx$ 학습률**: 기울기 방향이 일관되면 m 과 v 의 제곱근이 거의 같아져 비율이 약 1 → 스텝이 대략 학습률 크기로 상한이 잡힌다. 방향이 왔다 갔다 하면 m 은 상쇄되어 작아지고 v 는 그대로 → 비율이 1보다 작아져 **자동 감속**. 즉 "믿을 수 있는 방향엔 크게, 불확실한 방향엔 작게."
- **손실 스케일 불변**: 손실에 10을 곱해도 m 과 v 의 제곱근이 둘 다 10배라 약분 → 갱신 불변. SGD는 스텝이 10배로 커져 학습률을 다시 튜닝해야 한다.
- **파라미터별 학습률**: 드물게 갱신되는 파라미터(임베딩)는 v 가 작아 큰 스텝, 자주 갱신되는 것은 작은 스텝. AdaGrad 계보.
- **Adam = 모멘텀 + RMSProp + 편향 보정**. RMSProp에 없던 것이 1차 모멘트와 편향 보정.

• **알려진 이슈와 후속:**

- 수렴 반례(Reddi 2018) → AMSGrad(v 의 최댓값 유지).
- 초반엔 v 추정엔 쓰인 샘플이 적어 적응 학습률이 불안정 → **warmup** 필요(RAdam이 이론화). 트랜스포머에서 warmup이 사실상 필수인 이유.
- 비전 CNN에서 SGD+모멘텀보다 일반화가 못한 사례(Wilson 2017). NLP/트랜스포머에선 Adam(W)이 압도.
- "아주 작은 수"(epsilon)는 수치 안정용이지만 키우면(10의 -6~-4제곱) 안정성이 올라감(특히 반정밀도).
- **AdamW**(§10)가 weight decay 문제를 고친 현재 표준.

🔍 **꼬리 질문과 대응 포인트:**

• **Q1. "학습이 수천 스텝 진행되면 편향 보정 항은 어떻게 되나요?"**

- 🐼 감쇠율이 1보다 작으니 스텝 수 제곱을 하면 0으로 수렴 → "1 빼기 그 값"은 1이 됨 → 보정 효과가 자연스럽게 사라지고 순수 이동평균으로 안정.

- **Q2. "편향 보정을 빼면 구체적으로 무슨 일이?"**
 - 🖱️ v 가 m 보다 훨씬 심하게 축소(1000배 vs 10배)되어 비율이 초반 약 3배 과대 → 첫 스텝들이 너무 커서 불안정. "멈추는" 게 아니라 "튀는" 것.
- **Q3. "손실 함수를 10배 곱하면 Adam은?"**
 - 🖱️ 비율이라 약분되어 똑같이 학습. SGD는 스텝이 10배.
- **Q4. "Adam에 왜 warmup을 쓰나요?"**
 - 🖱️ 초반엔 v 추정치가 불안정해 적응 학습률이 크게 흔들림. warmup으로 v 가 안정될 때까지 학습률을 작게 유지.

📖 용어를 말로 풀면:

- **지수이동평균:** 최근 값에 큰 가중치, 과거는 기하급수적으로 잊는 평균. 감쇠율 0.9면 대략 최근 10개, 0.999면 최근 1000개 평균에 해당.
- **적응형 학습률:** 모든 파라미터에 같은 보폭을 주지 않고, 지형 경사에 따라 파라미터마다 보폭을 조절.
- **초기 편향:** 메모리를 0에서 시작해 초반 계산이 실제보다 0 쪽으로 작게 나오는 현상.
- **Warmup:** 학습 초반 몇 천 스텝 동안 학습률을 0에서 목표값까지 서서히 올리는 스케줄.

10. AdamW의 핵심 개선점 (Decoupled Weight Decay)

질문: AdamW는 무엇을 개선한 것인가요?

🎯 반드시 입에 올려야 할 키워드:

- ☐ **SGD에서의 일치:** SGD에서는 L2 규제와 weight decay가 수학적으로 완전히 같음
- ☐ **Adam에서의 왜곡:** 손실에 L2를 넣으면 규제항까지 2차 모멘트로 나뉘어 감쇠가 제대로 안 됨
- ☐ **Decoupled Weight Decay:** 감쇠를 기울기 계산에서 완전히 분리해 갱신 마지막에 직접 적용
- ☐ **결과:** 트랜스포머(BERT, GPT, ViT) 일반화 향상 + 학습률과 감쇠 계수의 튜닝 분리

💡 핵심 원리 (세금 감면 버그):

- **weight decay의 목적:** "파라미터 덩치가 크면 과적합 → 재산(가중치 크기)이 많은 놈에게 무거운 세금을 매겨 깎자."
- **기존 Adam의 버그:**
 - SGD에서는 손실에 가중치 제곱을 더하고 미분하면 자연스럽게 가중치가 깎였다.
 - Adam은 기울기를 "최근 변화량 크기(v 의 제곱근)"로 나누는 브레이크가 있다. 손실에 L2를 넣으면 우리가 때리려던 세금마저 그 브레이크로 나뉜다.
 - 자주 쓰이는 중요한 가중치는 최근 갱신이 많아 분모가 크다 → 세금이 100분의 1로 줄어 정작 재산 많은 놈의 세금이 깎이는 역전.
- **AdamW의 해결 (Decoupling):**
 - Loshchilov & Hutter: "세금 계산을 기울기 계산과 섞지 말고 완전히 분리하자."
 - Adam의 모멘텀 갱신은 순수 데이터 오차로만 계산하고, 갱신 식 맨 끝에서 가중치의 일정 비율을 직접 떼어낸다.
 - 덩치 큰 파라미터는 의도대로 시원하게 깎이고, 작은 파라미터는 적게 깎이는 정규화가 복원.
 - ✅ **정정:** 현대 거대 언어 모델은 대부분 AdamW(GPT-3, Llama, BERT). 단, T5/PaLM은 Adafactor, 일부 최신 모델(Kimi K2 등)은 Muon 같은 대안 → "100%"가 아니라 "사실상 표준".

🧠 심화 지식 (답변 재료):

- **SGD에서 L2 = weight decay인 이유 (말로 한 줄):** 손실에 "가중치 제곱의 절반 $\times$ 람다"를 더하면 기울기에 "람다 $\times$ 가중치"가 붙는다. 그걸 학습률로 곱해 빼면 "가중치를 $(1 - \text{학습률} \times \text{람다})$ 배로 줄이고 나서 기울기 방향으로 이동"과 똑같다. 즉 **동일한 것의 두 표현**.
- **Adam에서 깨지는 지점:** L2 방식은 "람다 $\times$ 가중치"가 기울기에 붙어 **m과 v에 함께 들어간다**. 그러면 갱신 때 v의 제곱근으로 나뉘니, 기울기 이력이 큰 파라미터일수록 감쇠가 **약해진다**. 감쇠가 가중치 크기에 비례하지 않게 됨. 추가로 모멘텀 m에 섞여 감쇠가 지연되고 방향과 얽힌다.
- **AdamW 갱신 (말로):** 파라미터 = 이전 파라미터 - 학습률 $\times$ (보정된 m $\div$ 보정된 v의 제곱근) - **학습률 $\times$ 람다 $\times$ 이전 파라미터**. 마지막 항이 분리된 감쇠. PyTorch 구현에서는 감쇠에 학습률이 곱해지므로 학습률 스케줄이 감쇠에도 같이 적용된다.
- **실무 함정:** PyTorch `Adam(weight_decay=)`는 L2 방식(구식), `AdamW(weight_decay=)`가 decoupled. 이름만 다른 게 아니라 동작이 다르다. 같은 숫자를 넣어도 효과가 다름.
- **논문의 두 번째 기여 — 하이퍼파라미터 분리:** L2-Adam에서는 최적 람다가 학습률에 강하게 의존해 둘을 동시에 그리드 서치해야 했다. AdamW에서는 최적 람다가 학습률과 거의 독립 $\rightarrow$ 튜닝이 쉬워짐. 코사인 스케줄 + warm restart와 결합해 발표됨.
- **관례 값:** PyTorch AdamW 기본 0.01, LLM 학습은 0.1(GPT-3, Llama), BERT 0.01.
- **No-decay 파라미터의 정확한 이유 (Q1 재료):**
 - Bias, LayerNorm의 scale/shift는 저차원(전체 파라미터의 1% 미만)이라 과적합 기여가 미미.
 - LayerNorm의 scale을 0으로 끝면 활성화값 스케일이 붕괴 $\rightarrow$ 정규화 층의 역할 훼손.
 - **심화 (선택):** LayerNorm/BatchNorm 앞에 있는 가중치는 크기를 2배로 키워도 정규화가 다시 맞춰버려 출력이 똑같다. 그래서 여기에 weight decay를 걸어도 "모델이 단순해지는" 게 아니라 가중치 숫자만 작아지고 $\rightarrow$ 같은 학습률이 상대적으로 큰 걸음이 되는 **유효 학습률 조절** 효과만 남는다(Van Laarhoven 2017). 한 문장: "정규화 층 앞에서는 weight decay가 규제가 아니라 학습률 손잡이 노릇을 한다. 그래서 wd와 lr이 얽힌다."
 - 임베딩 테이블도 no-decay로 두는 구현이 많다(희소 갱신이라 감쇠가 과함).
- **AdamW 이후 (아는 척 한 줄):** Adafactor(2차 모멘트를 행/열로 쪼개 메모리 절약, T5), LAMB(대배치 BERT), Lion(부호 기반, 메모리 적음), Muon(행렬 파라미터에 직교화 갱신, 2025년 이후 일부 LLM). 그래도 기본값은 AdamW.

🔍 꼬리 질문과 대응 포인트:

- **Q1. "LLM 학습 시 Bias나 LayerNorm에 weight decay를 안 거는 이유는?"**
 - 🐼 저차원이라 과적합 주원인이 아니고, 강제로 0으로 감쇠시키면 정규화 층의 고유 기능이 깨져 수렴을 방해. 가중치 행렬에만 걸고 Bias/Norm은 제외.
- **Q2. "PyTorch에서 Adam의 weight_decay와 AdamW의 weight_decay는 뭐가 다른가요?"**
 - 🐼 전자는 L2 방식(기울기에 더해져 모멘트에 섞임), 후자는 decoupled(갱신 마지막에 직접 감쇠). 같은 값이라도 실제 규제 강도가 다르다.
- **Q3. "SGD에서는 왜 분리가 필요 없나요?"**
 - 🐼 SGD는 기울기를 나누는 적응형 분모가 없어 L2와 weight decay가 동일. 분리가 의미를 갖는 건 적응형 옵티마이저뿐.
- **Q4. "Weight decay를 키우면 항상 일반화가 좋아지나요?"**
 - 🐼 아니다. 너무 크면 underfitting. 정규화 층 앞 가중치에서는 규제가 아니라 유효 학습률을 바꾸는 효과라 일반화와 직결되지 않는다. 튜닝 대상.

📖 용어를 말로 풀면:

- **Weight Decay:** 매 학습마다 가중치 크기에 비례하는 벌금을 매겨 0 쪽으로 조금씩 꺾는 다이어트.
- **Decoupled Weight Decay:** 옵티마이저의 속도 조절 공식 안에 세금을 섞지 않고, 마지막에 독립적으로 징수하는 방식.

- **No-decay 파라미터:** 뼈대를 지탱하는 최소 장치(편향, 정규화 계수)에는 감쇠를 면제하는 실무 규칙.
- **하이퍼파라미터 분리:** 학습률을 바꿔도 최적 감쇠 계수가 그대로라 둘을 따로 튜닝할 수 있는 성질.

면접 질문 은행 (셀프 테스트용 — 답 없음, 키워드 힌트만)

사용법: 질문을 소리 내 읽고 30초~1분 안에 답한다. 힌트는 가리고 먼저 답한 뒤, 힌트의 키워드가 빠졌는지 확인한다. 막히면 해당 항목의 🧠 심화 지식으로 돌아간다.

난이도: ★ 기본 (반드시 즉답) / ★★ 심화 (한 단계 파고들) / ★★★ 압박·실무 (반례·상황 판단)

1. 스케일링과 모델

- ★ **Q1-1.** 스케일링이 필요한 모델과 필요 없는 모델을 나눠 말해보세요. 나누는 기준은?
 - 힌트: 거리·분산·규제·경사하강 vs 순서
- ★ **Q1-2.** 결정 트리 입력에 로그 변환을 하면 결과가 바뀌나요?
 - 힌트: 단조 변환, 분할 후보는 순서로만 결정
- ★★ **Q1-3.** 커널을 쓰지 않는 선형 SVM도 스케일링이 필요한가요?
 - 힌트: 큰 가중치 벌주기, 단위 바뀌면 다른 경계선
- ★★ **Q1-4.** Ridge/Lasso는 스케일링이 필요한가요? 순수 선형회귀는요?
 - 힌트: 벌금이 스케일 의존 / 공식으로 풀면 불변, 경사하강이면 속도
- ★★★ **Q1-5.** RF와 로지스틱 회귀를 스택킹하려 합니다. 스케일링을 어디에?
 - 힌트: 베이스 모델별로, 메타모델이 선형이면 필요
- ★★★ **Q1-6.** 트리에 제곱 변환을 하면 결과가 같나요? 음수가 있다면요?
 - 힌트: 순서 깨짐 → 불변 아님

2. 정규화 / 표준화 / 이상치

- ★ **Q2-1.** Min-Max와 Z-score를 말로 비교해 주세요. 결과 범위는?
 - 힌트: 0~1 고정 vs 평균 0·표준편차 1·범위 없음
- ★ **Q2-2.** 이상치가 있으면 Min-Max에서 어떤 일이? 숫자 예시로.
 - 힌트: 최댓값에 걸림, 99명이 0으로
- ★★ **Q2-3.** 그럼 Z-score를 쓰면 해결되나요?
 - 힌트: 평균·표준편차도 오염, "덜 나뉘 뿐", RobustScaler / 로그 / 클리핑
- ★★ **Q2-4.** 표준화하면 정규분포가 되나요?
 - 힌트: 직선 변환, 모양 불변, 비선형 변환 필요
- ★★ **Q2-5.** 표준화가 경사하강 수렴을 빠르게 하는 이유를 그림으로.
 - 힌트: 길쭉한 골짜기, 가파른 방향 기준 학습률, 조건수
- ★★★ **Q2-6.** 이미지 픽셀은 왜 보통 0~1 정규화를 쓰나요?
 - 힌트: 범위 유한·알려짐, 이상치 없음, 활성화 함수 입력
- ★★★ **Q2-7.** RobustScaler를 쓰면 이상치 자체는 어떻게 되나요?
 - 힌트: 제거 아님, 여전히 큰 값, 별도 처리

3. 결측치

- ★ **Q3-1.** 결측 메커니즘 세 가지를 예시와 함께.
 - 힌트: 동전 / 다른 컬럼 / 빈칸 자체
- ★ **Q3-2.** 평균 대체의 문제점 두 가지 이상.
 - 힌트: 분산 축소, 상관 약화, 표준오차 과소

- ★★ Q3-3. 행 삭제가 안전한 경우는?
 - 힌트: MCAR에서만, 표본 손실(20컬럼 5% → 36% 생존)
- ★★ Q3-4. 회귀 대체는 평균 대체보다 나은가요?
 - 힌트: 상관 과대(반대 왜곡), 분산 여전히 과소, 노이즈 추가/다중 대체
- ★★ Q3-5. MICE가 단순 대체보다 나은 본질적 이유는?
 - 힌트: 다중 대체, 불확실성 보존, Rubin's rules
- ★★★ Q3-6. MAR인지 MNAR인지 어떻게 확인하나요?
 - 힌트: 데이터로 볼가, 도메인 지식, 민감도 분석
- ★★★ Q3-7. 소득 컬럼 30%가 결측이고 MNAR이 의심됩니다. 어떻게?
 - 힌트: 지시자 컬럼 + 중앙값, 트리는 NaN 네이티브, train에서만 fit

4. 데이터 누수

- ★ Q4-1. 전체 데이터로 스케일러를 fit한 뒤 분할하면?
 - 힌트: 테스트 통계 유출, 낙관적 평가
- ★ Q4-2. fit과 transform의 차이, 테스트셋엔 어느 것만?
 - 힌트: 외우기 vs 적용, transform만
- ★★ Q4-3. 누수의 종류를 세 층위로.
 - 힌트: 전처리 / 타겟 / 분할(그룹·시간·중복)
- ★★ Q4-4. 타겟 인코딩 누수가 스케일러 누수보다 위험한 이유는?
 - 힌트: 정답 통계가 입력으로, 희소 카테고리 = 자기 정답, out-of-fold
- ★★ Q4-5. 교차검증에서 Pipeline이 누수를 막는 원리는?
 - 힌트: fold마다 복제·재fit, 전처리+모델 한 객체
- ★★★ Q4-6. SMOTE를 전체에 적용하고 분할하면?
 - 힌트: 합성 샘플 원본이 test에, 근접 중복, imblearn Pipeline
- ★★★ Q4-7. 시계열을 셔플해서 나누면 왜 안 되나요?
 - 힌트: 미래로 과거 예측, 시간순 분할, 이동평균 피쳐
- ★★★ Q4-8. 학습 성능은 좋는데 배포 후 떨어집니다. 누수 외에 의심할 것은?
 - 힌트: train-serving skew, 다른 전처리 통계, 분포 이동

5. 데이터 증강의 원리

- ★ Q5-1. 증강이 과적합을 줄이는 이유를 한 문장으로.
 - 힌트: 같은 이미지 두 번 안 봄, 암기 불가, 본질 특징
- ★ Q5-2. "불변성" 관점에서 증강이 하는 일은?
 - 힌트: 변환해도 같은 출력, 사전지식 주입
- ★★ Q5-3. CNN은 이미 이동에 강한데 왜 증강이 필요한가요?
 - 힌트: 합성곱 = 평행이동만, 회전·스케일·조명은 구조에 없음
- ★★ Q5-4. 증강이 Weight Decay와 같은 정규화라면 그냥 Weight Decay를 세게 걸면?
 - 힌트: 작용 대상 다름(데이터 vs 파라미터), 도메인 지식은 증강만
- ★★ Q5-5. Mixup의 원리와 효과, CutMix와의 차이는?
 - 힌트: 비율로 섞기·소프트 라벨·과신 억제 / 패치 교체·면적 비율·국소 특징
- ★★★ Q5-6. 증강을 너무 세게 걸면?
 - 힌트: 분포 이동, underfitting, 강도도 하이퍼파라미터
- ★★★ Q5-7. 표 형식 데이터에서 증강이 잘 안 되는 이유는?
 - 힌트: 자명한 불변성 없음, SMOTE 정도, 도메인 지식

6. 레이블 보존

- ★ Q6-1. 숫자에 180도 회전과 좌우 반전을 걸면?
 - 힌트: 6과 9, 거울 문자
- ★ Q6-2. 증강 기법을 고르는 기준을 한 문장으로.
 - 힌트: 사람이 봐도 같은 라벨, 실제 발생하는 변형
- ★★ Q6-3. 손글씨 숫자에 안전한 증강 4가지.
 - 힌트: 소각도 회전, 탄성 왜곡, 이동/스케일, 획 두께
- ★★ Q6-4. 흉부 X-ray, 도로 표지판, 화자 인식 — 각각 금지 증강은?
 - 힌트: 좌우 반전(심장) / 좌우 반전(화살표) / 피치 시프트
- ★★★ Q6-5. 임의 각도로 회전된 숫자를 인식해야 한다면?
 - 힌트: 라벨도 함께 바꾸는 증강, 방향 정규화, 보조 라벨
- ★★★ Q6-6. 객체 검출에서 좌우 반전 증강 시 라벨은?
 - 힌트: 박스 좌표도 반전, 등변성

7. 검증/테스트 증강과 TTA

- ★ Q7-1. 검증셋에 증강을 적용하나요? 정규화도 빠나요?
 - 힌트: 무작위 증강만 끄, 결정적 전처리는 동일 적용
- ★ Q7-2. 검증셋이 재현 가능해야 하는 이유는?
 - 힌트: early stopping·모델 선택 기준, 점수 노이즈
- ★★ Q7-3. TTA란 무엇이고 왜 성능이 오르나요?
 - 힌트: 입력 공간 앙상블, 흔들림 감소, soft voting
- ★★ Q7-4. TTA에 어떤 변환을 골라야 하나요?
 - 힌트: 학습에 쓴 라벨 보존 변환, 모델이 준불변인 것
- ★★ Q7-5. TTA의 비용과 적합/부적합 도메인은?
 - 힌트: N배 연산, 33밀리초 예산, 의료·오프라인 vs 실시간
- ★★★ Q7-6. 세그멘테이션에서 좌우 반전 TTA 출력은 어떻게 합치나요?
 - 힌트: 출력 다시 반전 후 평균
- ★★★ Q7-7. ImageNet-C처럼 테스트셋을 일부러 손상시키는 건 "테스트에 증강" 아닌가요?
 - 힌트: 강건성이라는 별도 지표

8. Batch / SGD / Mini-batch

- ★ Q8-1. 세 방식을 "분산"과 "비용" 두 축으로 비교.
 - 힌트: 흔들림은 배치에 반비례, 스텝 비용은 배치에 비례, 업데이트 횟수
- ★ Q8-2. 미니배치가 표준이 된 이유는?
 - 힌트: GPU 병렬, 메모리, 적당한 노이즈
- ★★ Q8-3. SGD의 기울기가 매번 틀린데 어떻게 수렴하나요?
 - 힌트: 평균적으로 맞음, 근처에서 진동, 학습률 감소 스케줄
- ★★ Q8-4. 배치를 극단적으로 키우면 일반화가 나빠지는 이유는? 반론은?
 - 힌트: sharp minima, 노이즈 = 온도 / Linear scaling + warmup, 스텝 수 부족
- ★★ Q8-5. 배치를 2배로 키우면 학습률은?
 - 힌트: 선형 스케일링, warmup
- ★★★ Q8-6. 배치를 키워도 더 빨라지지 않는 지점이 있나요?
 - 힌트: critical batch size
- ★★★ Q8-7. 배치 4로 BatchNorm 모델을 학습하면?
 - 힌트: 배치 통계 불안정, GroupNorm/LayerNorm
- ★★★ Q8-8. 왜 매 에포크 셔플하나요?

- 힌트: 배치 편향, 클래스 몰림

9. Adam

- ★ Q9-1. Adam의 갱신 절차를 말로 설명하고 각 항의 역할은?
 - 힌트: m 방향·관성 / v 크기·적응 학습률 / 보정 / 아주 작은 수
- ★ Q9-2. 편향 보정이 왜 필요한가요?
 - 힌트: 0에서 시작, 초반엔 일부만 채워짐
- ★★ Q9-3. 보정을 빼면 구체적으로 어떤 증상?
 - 힌트: v가 더 심하게 축소, 비율 약 3배, 스텝 폭주 (멈춤 아님!)
- ★★ Q9-4. 스텝이 많이 쌓이면 보정 항은?
 - 힌트: 감쇠율의 거듭제곱이 0으로, 보정 사라짐
- ★★ Q9-5. 손실 함수에 10을 곱하면 Adam과 SGD는 각각?
 - 힌트: Adam 불변(약분) / SGD 스텝 10배
- ★★ Q9-6. Adam에서 warmup을 쓰는 이유는?
 - 힌트: 초반 v 추정 불안정
- ★★★ Q9-7. 기울기 방향이 계속 바뀌는 파라미터는 Adam에서?
 - 힌트: m이 상쇄되어 작아짐, 자동 감속
- ★★★ Q9-8. Adam이 SGD보다 일반화가 나쁘다는 말, 어디서?
 - 힌트: 비전 CNN 사례, NLP는 Adam 우세

10. AdamW

- ★ Q10-1. AdamW가 고친 문제를 한 문장으로.
 - 힌트: 감쇠가 v로 나뉘어 왜곡, 분리
- ★ Q10-2. SGD에서 L2 규제와 weight decay가 같다는 걸 말로 설명.
 - 힌트: 기울기에 람다×가중치 → 가중치를 일정 비율로 줄이는 것과 동일
- ★★ Q10-3. Adam에서는 왜 같지 않나요?
 - 힌트: 규제항이 m, v에 섞임, 분모로 나뉘면, 큰 v = 약한 감쇠
- ★★ Q10-4. AdamW 갱신을 말로.
 - 힌트: 마지막에 학습률×람다×가중치를 따로 뺌
- ★★ Q10-5. PyTorch Adam의 weight_decay와 AdamW의 차이는?
 - 힌트: L2 방식 vs decoupled, 같은 값이라도 효과 다름
- ★★ Q10-6. Bias/LayerNorm에 weight decay를 안 거는 이유는?
 - 힌트: 저차원, scale이 0으로 가면 붕괴, (정규화 앞은 유효 학습률 조절)
- ★★★ Q10-7. AdamW 논문의 "하이퍼파라미터 분리"란?
 - 힌트: 최적 람다가 학습률에 독립, 튜닝 용이
- ★★★ Q10-8. Weight decay를 키우면 항상 일반화가 좋아지나요?
 - 힌트: underfitting, 정규화 앞에서는 규제 아님

종합 · 교차 질문 (여러 항목을 엮어야 답이 되는 것)

- ★★ X-1. KNN 대체를 쓸 때 주의할 점은? (§1 + §3 + §4)
 - 힌트: 거리 → 스케일링 먼저, train에서만 fit, 비용
- ★★ X-2. 검증셋에서 "하면 안 되는 것" 목록을 만들어 보세요. (§4 + §7)
 - 힌트: fit 호출, 무작위 증강, 미래 정보 / 결정적 전처리는 해야 함
- ★★ X-3. "노이즈가 정규화 역할을 한다"를 두 사례로. (§5 + §8)
 - 힌트: 증강 = 데이터 노이즈, 미니배치 = 기울기 노이즈

- ★★★★★ **X-4.** 표준화 없이 학습이 느린 모델을 Adam으로 바꾸면 해결되나요? (§2 + §9)
 - 힌트: 파라미터별 적응 학습률이 부분 완화, 완전 대체는 아님
- ★★★★★ **X-5.** 트랜스포머 학습 레시피에서 AdamW·warmup·no-decay·큰 배치가 각각 왜? (§8 + §9 + §10)
 - 힌트: decoupled 감쇠 / 초반 v 불안정 / LayerNorm 보호 / linear scaling
- ★★★★★ **X-6.** 캐글에서 검증 점수는 좋았는데 리더보드가 낮습니다. 의심 순서는? (§3 + §4 + §7)
 - 힌트: 누수(타겟 인코딩·그룹·시간), 검증 분할 방식, 대체/스케일 fit 위치, TTA 차이

항목 간 연결 지도 (답변 중 자연스럽게 엮을 수 있는 고리)

- §1 ↔ §2: "SVM은 스케일 민감(§1) → 그럼 어떤 스케일러?(§2)"
- §1 ↔ §3: KNN 대체는 거리 기반이라 스케일 민감.
- §2 ↔ §8/§9: 길쭉한 골짜기(조건수) → 경사하강 지그재그 → Adam의 파라미터별 적응 학습률이 부분 완화.
- §3 ↔ §4: 결측 대체 통계량도 train에서만 fit(누수).
- §4 ↔ §7: 검증셋은 "배포 분포 모사"가 목적 → 누수도 무작위 증강도 금지, 결정적 전처리는 동일 적용.
- §5 ↔ §6 ↔ §7: 증강의 원리 → 허용 변환 기준 → 평가 시 처리와 TTA.
- §5 ↔ §8: 증강(데이터 노이즈)과 미니배치(기울기 노이즈) 모두 "노이즈가 정규화"라는 공통 주제.
- §8 ↔ §9: SGD 노이즈 vs Adam의 자동 감속(방향이 흔들리면 스텝이 작아짐).
- §9 ↔ §10: Adam의 v 분모가 weight decay를 왜곡 → AdamW.

참고 문헌 (References)

1. [Q1. SVM & Random Forest 스케일링]
 - Cortes & Vapnik (1995). Support-vector networks. *Machine Learning*.
 - Breiman (2001). Random forests. *Machine Learning*.
2. [Q2. 표준화/정규화 및 경사하강법 기하학]
 - LeCun, Bottou, Orr & Müller (2012). Efficient backprop. *Neural Networks: Tricks of the Trade*.
 - Rousseeuw & Hubert (2011). Robust statistics for outlier detection. *WIREs DMKD*.
3. [Q3. 결측치 메커니즘 및 MICE]
 - Rubin (1976). Inference and missing data. *Biometrika*.
 - Little & Rubin (2019). *Statistical analysis with missing data* (3rd ed.). Wiley.
 - van Buuren & Groothuis-Oudshoorn (2011). mice. *Journal of Statistical Software*.
4. [Q4. 데이터 누수 및 파이프라인]
 - Kaufman, Rosset, Perlich & Stitelman (2012). Leakage in data mining. *ACM TKDD*.
 - Pedregosa et al. (2011). Scikit-learn. *JMLR*.
 - Hastie, Tibshirani & Friedman (2009). *The Elements of Statistical Learning*, §7.10.2.
5. [Q5 & Q6. 데이터 증강]
 - Bishop (1995). Training with noise is equivalent to Tikhonov regularization. *Neural Computation*.
 - Chapelle, Weston, Bottou & Vapnik (2000). Vicinal risk minimization. *NeurIPS*.
 - LeCun, Bottou, Bengio & Haffner (1998). Gradient-based learning applied to document recognition. *Proc. IEEE*.
 - Simard, Steinkraus & Platt (2003). Best practices for CNNs applied to visual document analysis. *ICDAR*.
 - Zhang, Cisse, Dauphin & Lopez-Paz (2017). mixup. *arXiv:1710.09412*.
 - Yun et al. (2019). CutMix. *ICCV*.
6. [Q7. TTA]

- Krizhevsky, Sutskever & Hinton (2012). ImageNet classification with deep CNNs. *NeurIPS*.
- Simonyan & Zisserman (2014). Very deep convolutional networks. *arXiv:1409.1556*.

7. [Q8. 경사하강법 분산과 Sharp vs Flat Minima]

- Bottou (2010). Large-scale machine learning with SGD. *COMPSTAT*.
- Keskar et al. (2016). On large-batch training for deep learning. *ICLR*.
- Goyal et al. (2017). Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv:1706.02677*.
- Hoffer, Hubara & Soudry (2017). Train longer, generalize better. *NeurIPS*.
- McCandlish, Kaplan, Amodei et al. (2018). An empirical model of large-batch training. *arXiv:1812.06162*.
- Dauphin et al. (2014). Identifying and attacking the saddle point problem. *NeurIPS*.

8. [Q9. Adam]

- Kingma & Ba (2014). Adam: A method for stochastic optimization. *ICLR*.
- Reddi, Kale & Kumar (2018). On the convergence of Adam and beyond. *ICLR*.
- Liu et al. (2020). On the variance of the adaptive learning rate and beyond (RAdam). *ICLR*.

9. [Q10. AdamW]

- Loshchilov & Hutter (2017). Decoupled weight decay regularization. *ICLR*.
- Van Laarhoven (2017). L2 regularization versus batch and weight normalization. *arXiv:1706.05350*.